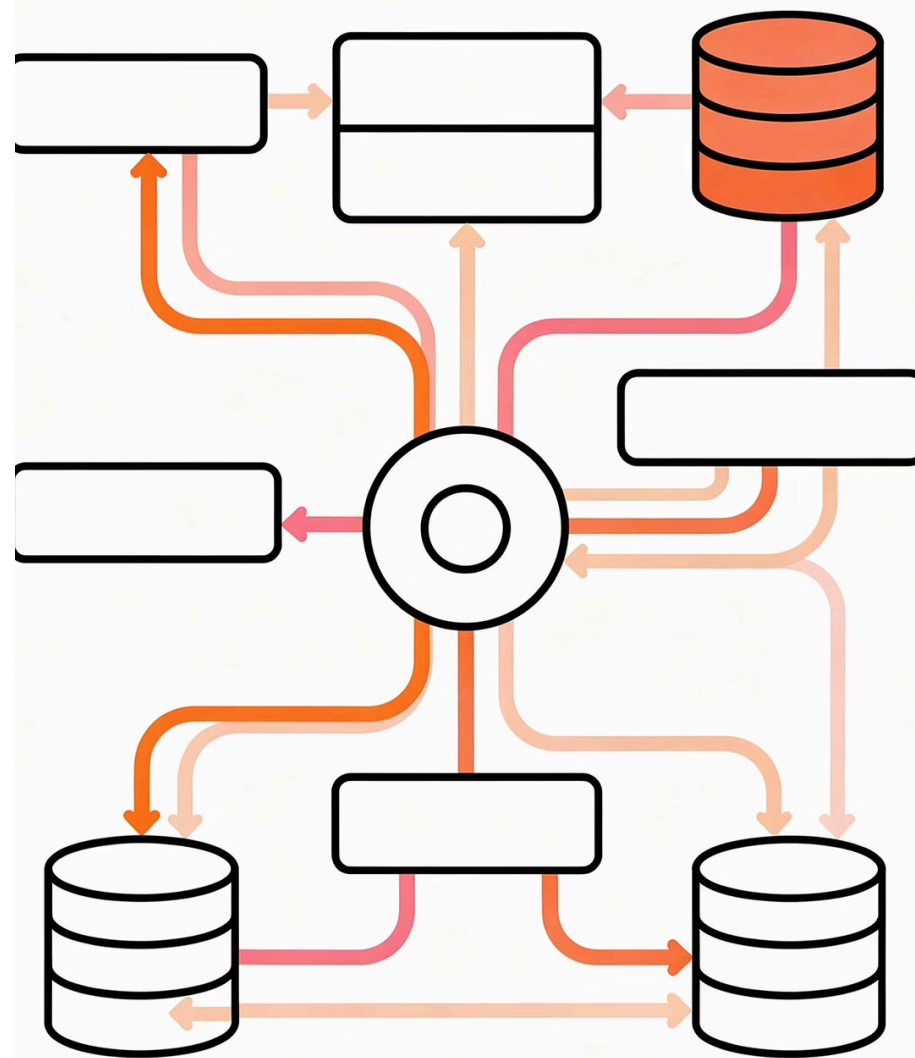


WEEK 3 · FEB 2026

Database After Graduation

A practical, engineer-first guide to pagination, API lifecycles, connection pooling, ORM vs. raw SQL, and everything in between — written for computer science graduates stepping into real backend systems.

By **Narges Yarahmadi**



What We Cover This Week

Six high-impact topics that bridge the gap between textbook databases and production-grade backend engineering.

01

Pagination Strategies

Offset-based vs. cursor-based pagination — when each strategy wins and why getting this wrong kills performance at scale.

03

API Request Lifecycle

The full 8-step journey of a single HTTP request — from TCP bytes to JSON response — including middleware and observability.

05

Pool Sizing & Failure Modes

How to calculate the right pool size, recognize common failure modes, and think about async vs. sync trade-offs.

02

Databases in Backend Architecture

Where the database lives in a modern API system and how data moves safely from client to storage and back.

04

Access Patterns & Connection Pooling

The three core database access patterns, common anti-patterns, and how pooling transforms a demo app into a production system.

06

ORM vs. Raw SQL & Swagger

When to use each data-access strategy in production, and how FastAPI's Swagger UI gives you instant interactive API documentation.

Pagination Strategies

Pagination is not a UI concern — it is a database performance concern disguised as a UI feature. Imagine an Uber-like app with 5 million ride records. Returning all of them would cause massive memory usage, network congestion, slow rendering, and likely a timeout. We always return a *page*, not everything.

Without Pagination

- Massive memory spikes
- Network congestion
- Slow frontend rendering
- Possible request timeouts

With Pagination

- Controlled memory usage
- Faster network transfers
- Better user experience
- Predictable DB load

Two Main Strategies

- Offset-based pagination
- Cursor-based pagination

Each has a specific use case. Choosing the wrong one degrades your system under growth.

Offset-Based

```
SELECT * FROM rides ORDER BY created_at DESC LIMIT 20 OFFSET 40;
```

Simple and intuitive. Works great for small, stable datasets. But at high offsets, the DB still *walks* through every prior row — it does not jump. At OFFSET 1,000,000, that is expensive.

Cursor-Based

```
SELECT * FROM rides WHERE created_at < '2025-02-01 10:00:00'
ORDER BY created_at DESC LIMIT 20;
```

Industrial-grade. Uses a position marker instead of row counting. Stable under inserts — new rows do not disturb already-fetched pages. The engine behind Twitter timelines, Instagram feeds, and ride-history in large apps.

1

Use Offset When

Dataset is small, you need visible page numbers, building admin dashboards or internal tools.

2

Use Cursor When

Dataset is large, data changes frequently, implementing infinite scroll, or serving high-traffic systems.

3

Advanced: Security

Always cap client-provided limits: `limit = min(requested_limit, 100)`. Never trust raw input — a rogue `limit=1000000` can cripple your database.

4

Advanced: Indexing

Always paginate on indexed columns. Compound filters like `WHERE user_id = 42 AND status = 'COMPLETED' AND created_at < ?` require indexes that support both filtering and ordering.

📌 **Mental Model:** Offset says "skip and return." Cursor says "continue from here." Offset is counting. Cursor is positioning. Industrial systems prefer positioning. Good pagination is invisible when done right — bad pagination shows up as slow endpoints, rising latency with page number, and user complaints about missing data.

Databases in Backend Architecture

Stop thinking about databases as tables. Start thinking about them as one stateful component in a larger, stateless system. Users never talk directly to a database — they talk to APIs, and the API mediates every interaction.

The Core Architecture

Client → API Server → Database

The client can be a web browser, a mobile app, or another backend service. The API server receives HTTP requests, validates input, executes business logic, queries the database, and returns a JSON response. The database stores persistent data and ensures consistency.

Key insight: The API is *stateless* — it does not remember previous requests. The database is *stateful* — it holds long-term data. This separation is fundamental to every production system.

One Request, End to End

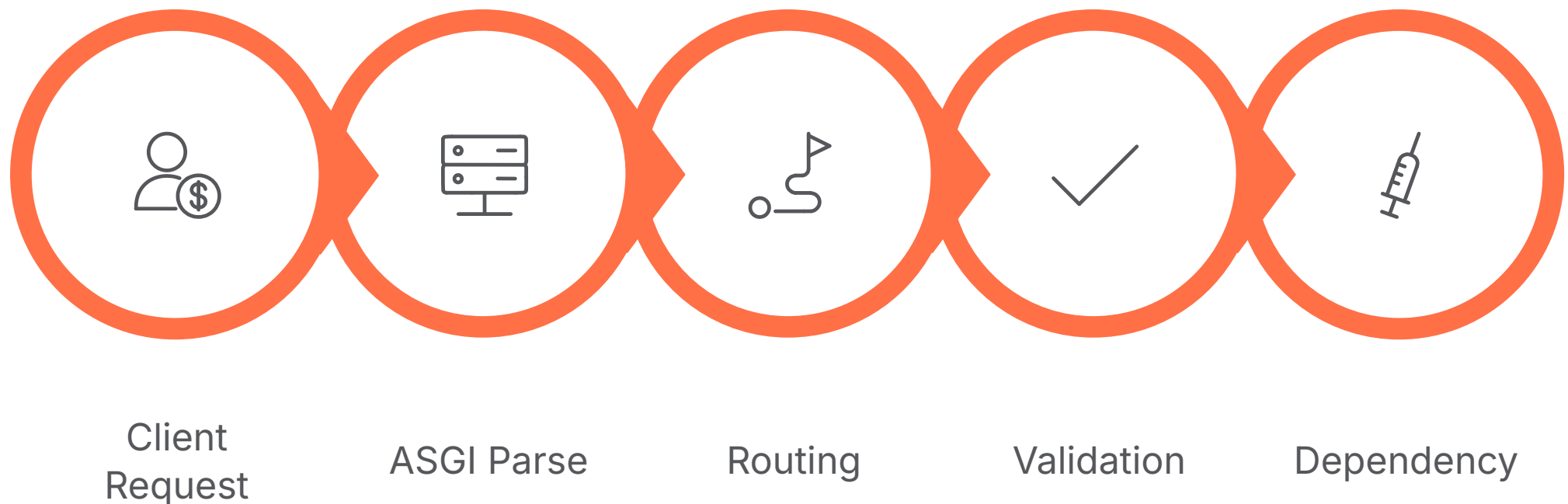
Follow a single `GET /users/42` request internally:

1. Client sends HTTP request over TCP
2. FastAPI receives and parses the request
3. Router matches the path to a handler
4. Dependency injection creates a DB session
5. Business logic executes
6. SQL query runs against the database
7. Database returns rows
8. Data is serialized to JSON and returned

Each request creates work for the database. At high concurrency, the database becomes the bottleneck — which is exactly why connection pooling and query design matter.

The API Request Lifecycle

An API request lifecycle is a **controlled transformation pipeline**: raw network bytes → structured request → validated input → domain logic → database interaction → structured output → network bytes again. Each stage has exactly one responsibility. When responsibilities blur, systems become fragile.



Every performance issue, every security issue, and every scaling problem in backend systems happens somewhere along this pipeline. Understanding it deeply means you can diagnose real-world problems instead of just writing endpoints.

Routing & Validation (Steps 3–4)

Routing is pure pattern matching — no business logic. FastAPI validation happens before your function runs. Sending `GET /users/abc` when `user_id` is an `int` returns a 422 immediately. This is one of the strongest defensive mechanisms against bugs and attacks.

Dependency Injection & Pooling (Step 5)

FastAPI calls `get_db()` before your function. Opening a fresh DB connection per request would destroy performance. A connection pool reuses existing connections — request arrives, borrow from pool, execute, return to pool. Efficient and scalable.

Serialization & Security (Step 7)

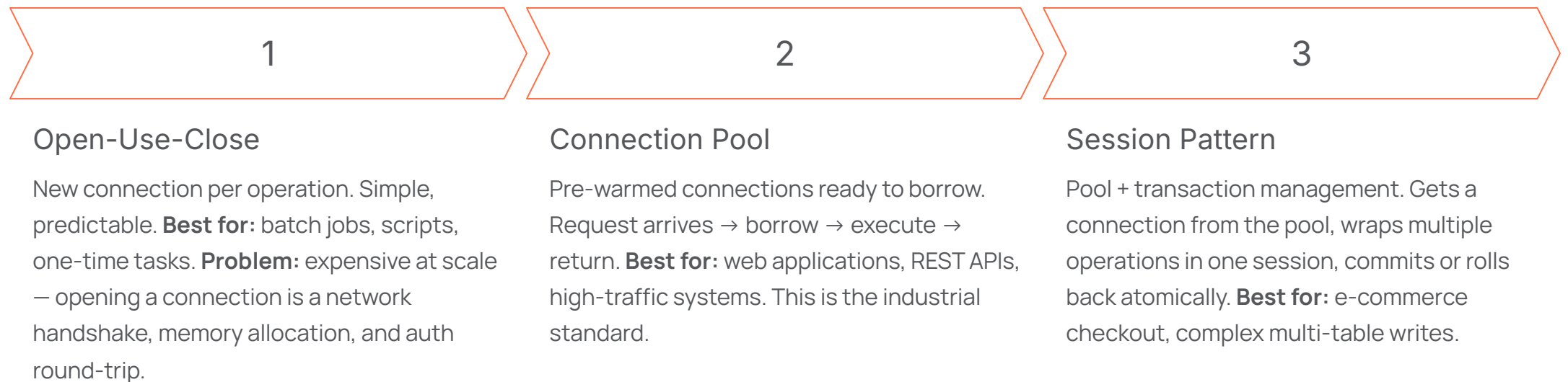
Python objects cannot travel over HTTP. Pydantic converts them to JSON and also filters fields — a `password_hash` in your DB model is never exposed unless explicitly allowed. Serialization is both a format concern and a security boundary.

Middleware & Observability

Middleware runs before and after endpoints: logging, rate limiting, JWT authentication, security headers. Without observability — tracking latency, error rates, query time, throughput — scaling is guesswork. In production, you cannot fix what you cannot measure.

Access Patterns & Connection Pooling

When your API talks to a database, it is having a conversation. Database Access Patterns are the standardized protocols for that conversation. Think of it like a library: the library is your database, the librarian is the connection, you are the application, and the book is the data. *Different ways of asking for books = different access patterns.*



Common Anti-Patterns — What NOT to Do

✗ Anti-Pattern 1: Connection Leak

Opening a connection and never closing it. The pool exhausts silently. Fix: always use `try/finally` or a context manager to guarantee `connection.close()` is called.

✗ Anti-Pattern 2: Death by Connections

Creating a brand-new `mysql.connector.connect()` inside every endpoint handler. Your database hits its connection limit (~1000) and crashes. Fix: always use a shared connection pool via `Depends(get_db)`.

✗ Anti-Pattern 3: Transaction Turtle

Calling `session.commit()` inside a loop — one commit per item. This is catastrophically slow for bulk operations. Fix: add all items in the loop and issue a single `session.commit()` at the end.

✓ The Industry Standard (FastAPI)

Define `get_db()` as a generator that yields a session and closes it in `finally`. Use `Depends(get_db)` in every endpoint. One session per request. Always close. Always pool.

Batch Job / Script

Use **Open-Use-Close**. Runs once, no need for a pool.

High-Traffic API

Use **Connection Pool**. Must handle many concurrent requests efficiently.

E-commerce Checkout

Use **Session Pattern**. Multiple tables, atomicity required.

Real-time Dashboard

Use **Connection Pool**. Many small, frequent queries need fast borrow/return cycles.

Connection Pool Sizing — The Quiet Force

Pool sizing is one of the quiet forces that determines whether your API survives production traffic or collapses under it. A database connection is a network socket, memory allocation, authentication handshake, session state, and server-side resource reservation. Reuse — not recreation — is the production mindset.

The Sizing Formula

You do not guess pool size — you calculate it:

$$\text{Concurrency} \approx \text{request_rate} \times \text{query_time}$$

Example: 200 requests/sec × 0.05 sec average query = **10 concurrent connections needed**. Not 100. Not 200. Around 10.

In SQLAlchemy:

```
engine = create_engine(URL, pool_size=10, max_overflow=20)
```

`max_overflow` allows temporary burst capacity. But unlimited overflow is dangerous — always bound it.

Multiplying Across Instances

If you deploy 3 containers each with `pool_size=15`, your total potential connections = **45**. MySQL's default `max_connections = 100`. Three services at this size would consume 45 slots — but add a fourth service and you approach the limit fast.

Always size with the full deployment topology in mind. This is where many production outages begin: developers size per instance and forget total capacity.

For tighter control at scale, external poolers like **PgBouncer** sit between the app and DB, managing connections across many microservices.

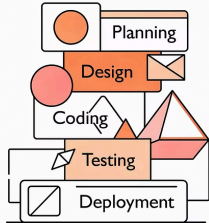
Common Failure Modes

Pool Too Small Requests hang. High latency. Threads queue for connections. DB CPU looks low — because the bottleneck is the pool, not the database. Fix: increase pool size or reduce query time.	Pool Too Large DB CPU spikes. Context switching overhead. Query times rise. MySQL shows too many active sessions thrashing each other. More connections ≠ more speed when DB cores are saturated.	Async vs. Sync Synchronous APIs hold a connection while waiting for I/O, requiring larger pools. Async APIs release connections faster and can operate with smaller pools — but long transactions still block others. Keep DB interactions short, commit quickly, release early.
---	---	--

📌 **The deeper insight:** A connection pool is a *concurrency throttle*. It protects the database from being overwhelmed by the application. When the database collapses, the entire system collapses. Pool sizing is not about maximizing performance — it is about **controlling pressure predictably**. Start small, measure, adjust. Scaling is a feedback loop, not a one-time configuration.

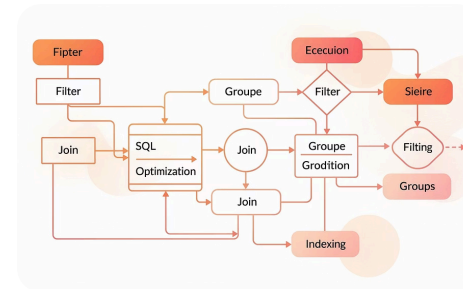
ORM vs. Raw SQL

ORM (Object Relational Mapping) is a layer between your application code and the database. It maps **tables** → **classes**, **rows** → **objects**, and **columns** → **attributes**. Instead of writing `SELECT * FROM users WHERE id = 5`, you write `User.query.filter_by(id=5).first()` and let the framework generate SQL. But this is not a question of which is better — it is a question of *when to use which*.



ORM Advantages

- **Developer productivity:** write less code, think in objects
- **Maintainability:** schema changes are centralized in model classes
- **Security:** automatic parameter escaping reduces SQL injection risk
- **Abstraction:** switch from MySQL to PostgreSQL with minimal changes



Raw SQL Advantages

- **Full control:** write exactly the query you want
- **Complex queries:** window functions, aggregations, complex joins
- **Predictable performance:** no hidden query generation
- **Index & plan control:** fine-tune for analytics and reporting

The Industry Reality

Use ORM When...

- Building APIs with standard CRUD operations
- Developing rapidly (startups, MVPs)
- Working in large teams that need consistency
- Building admin dashboards

Use Raw SQL When...

- Building analytics or reporting systems
- Performance-critical paths (trading, real-time)
- Large-scale data processing
- Complex query tuning is required

Most production systems use ORM for ~80% of operations and drop to raw SQL for the heavy parts. **The engineering mindset:** ORM is a tool. SQL is a language. You must know both. Using ORM does not remove the need to understand indexes, joins, and execution plans — it just hides them. If you don't understand them, ORM won't save you.

What is Swagger?

Swagger is an interactive API documentation interface that FastAPI generates *automatically* from your code. The moment you write `@app.get("/products")`, FastAPI registers the route, documents it, and builds a UI for testing it — no Postman required. Swagger is a control panel for your API.

Try It Step by Step

After running `uvicorn`:

1. Open `http://127.0.0.1:8000/health` — you should see `{"status":"ok","db":1}`. If yes, your DB connection is working.
2. Open `http://127.0.0.1:8000/docs` — the Swagger UI appears with all your routes listed.
3. Click any endpoint (e.g., `GET /products`), click **Try it out**, fill any parameters, and click **Execute**.
4. You will see the exact request URL, the JSON response body, and the HTTP status code.

That JSON is literally your database speaking through Python.

What Happens Behind the Scenes

When you call `GET /products?category=phone&limit=5`:

1. FastAPI parses query parameters
2. Calls your Python handler function
3. SQLAlchemy generates and runs a MySQL query
4. Results are converted to Python dicts
5. Pydantic serializes to JSON
6. Swagger displays the response

FastAPI auto-generates the OpenAPI schema that powers Swagger from your type annotations and Pydantic models — meaning your documentation is always in sync with your code. No separate docs to maintain.

Key Takeaways

Six principles that separate engineers who write endpoints from engineers who build systems.



Pagination is a Performance Contract

Use cursor-based pagination for high-traffic, growing datasets. Use offset for small, stable ones. Always index your ordering column and cap client-provided limits.



The API is Stateless; the DB is Stateful

Every request creates DB work. Understanding the 8-step lifecycle lets you diagnose performance and security issues at their root — not just treat symptoms.



Always Pool in Web Apps

One session per request. Always close connections. Match your pattern to the use case: pool for APIs, session for transactions, open-use-close for scripts.



Size Your Pool with Math

$\text{Concurrency} \approx \text{request_rate} \times \text{query_time}$. Start small, measure, adjust. Multiply across all deployed instances. A pool is a throttle — it protects the DB, not just the app.



ORM + SQL, Not ORM vs. SQL

80% ORM for productivity, 20% raw SQL for power. Understanding indexes, joins, and execution plans is non-negotiable — ORM does not hide you from the database forever.



Swagger is Your Interface

FastAPI auto-generates interactive docs from your code. Use it to test, debug, and validate that your API behaves exactly as designed — your documentation is always live.

📌 **The Real Engineering Mindset:** Backend engineering is not about going fast. It is about **surviving load predictably**. Every design decision — pagination strategy, pool size, access pattern, ORM vs. SQL — is a trade-off between speed, safety, and scalability. The engineers who internalize these trade-offs are the ones who build systems that last.